

Reglas de Asociación, Grafos y Analítica de Texto

Semana 14

SECCIÓN 1

Reglas de Recomendación

Filtrado colaborativo

Sistemas de Recomendación

Tipos principales

Enfoque	Idea central
Basado en contenido	Recomienda ítems similares por atributos, según el perfil del usuario
Filtrado colaborativo	Usa preferencias de usuarios similares, sin conocer el contenido
Basado en conocimiento	Usa reglas explícitas de dominio (ej. requisitos técnicos)
Híbrido	Combina varios enfoques para mitigar sus debilidades individuales

(Tecnología BI, s.f.)

Filtrado Colaborativo

Basado en usuario vs. basado en ítem

User-based

- Encuentra usuarios **similares** al usuario objetivo
- Recomendando lo que gustó a esos usuarios
- Similitud: coseno, correlación de Pearson

Item-based

- Encuentra ítems **similares** a los que el usuario ya consumió
- Más estable en catálogos grandes con muchos usuarios

El filtrado colaborativo puede ser basado en vecindarios (similitud directa) o basado en modelos (factorización de matrices, embeddings latentes) (GraphEverywhere, s.f.; Manzano Chicano, s.f.).

SECCIÓN 2

NoSQL y Grafos

Modelos de datos NoSQL, bases de datos de grafos y algoritmos de redes sociales

Modelos de Datos NoSQL

Cuatro tipos principales

Tipo	Estructura	Ejemplos
Key-Value	Tabla hash de identificador → valor	Redis, DynamoDB
Documentos	Documentos JSON/BSON, anidables e indexables	MongoDB
Column-Family	Mapa multidimensional, disperso y distribuido	Cassandra
Grafos	Nodos y relaciones con propiedades	Neo4j

Document stores destacan en esquemas flexibles, key-value en caché ultrarrápida, column-family en alta escritura y grafos en relaciones complejas (BlazeClan, s.f.; AltexSoft, s.f.).

Criterios de Elección de NoSQL

Cuándo usar cada tipo

- **Alta frecuencia de escritura** → key-value o column-family (optimizados para throughput)
- **Esquema cambiante / datos semiestructurados** → documentos
- **Relaciones complejas y muy conectadas** → grafos
- **Consultas simples por clave** → key-value

No existe un modelo NoSQL universalmente superior: la elección depende del patrón de acceso a los datos, no solo del volumen (TechTarget, s.f.; MongoDB, s.f.).

Bases de Datos de Grafos

Modelo de propiedades y lenguaje Cypher (Neo4j)

Modelo de propiedades etiquetado

- **Nodos:** entidades (con etiquetas y propiedades)
- **Relaciones:** conexiones dirigidas entre nodos (con tipo y propiedades)
- Sin esquema rígido de relaciones predefinido

Cypher

Lenguaje declarativo tipo "ASCII art" para patrones:

```
MATCH (u:Usuario)-[:SIGUE]->(v:Usuario)
WHERE u.nombre = 'Ana'
RETURN v.nombre
```

Neo4j está especialmente indicado para modelar redes sociales y sistemas de recomendación (Blog IngcapaDev, s.f.; UOC, s.f.).

Redes Sociales como Grafos

Representación de nodos y relaciones

- **Nodos:** usuarios, publicaciones, grupos
- **Aristas:** "sigue a", "es amigo de", "le gusta", "comparte"
- Grafos pueden ser **dirigidos** (seguir) o **no dirigidos** (amistad mutua)
- Permiten preguntas imposibles de responder eficientemente en SQL relacional: "¿quiénes son amigos de mis amigos pero no míos?"

Modelar una red social como grafo hace explícita la estructura de conexión, base para calcular centralidad, comunidades y difusión de información (Cambridge Intelligence, s.f.).

Medidas de Centralidad

¿Qué nodos son más importantes?

Medida	Mide
Grado (degree)	Número de conexiones directas de un nodo
Cercanía (closeness)	Qué tan cerca está un nodo de todos los demás
Intermediación (betweenness)	Cuántas veces un nodo actúa como puente en caminos más cortos
PageRank	Importancia recursiva: un nodo es importante si apunta a él otros nodos importantes

Betweenness centrality identifica los nodos "puente" que conectan comunidades distintas en la red (ResearchGate, s.f.; Neo4j, s.f.).

Detección de Comunidades

Algoritmo de Louvain (modularidad)

Funcionamiento (bottom-up, multinivel)

1. Cada nodo empieza como su propia comunidad
2. Se agrupan iterativamente los nodos que **maximizan la ganancia de modularidad**
3. Cada comunidad resultante se colapsa en un **super-nodo**
4. Se repite el proceso sobre la red de super-nodos hasta que no hay más ganancia posible

Louvain destaca por su alta velocidad y buena calidad de resultados, aunque tiene un límite de resolución que dificulta detectar comunidades pequeñas dentro de redes muy grandes (GraphEverywhere, s.f.).

Conteo de Triángulos

Coeficiente de clustering

- Un **triángulo** ocurre cuando tres nodos están todos conectados entre sí
- Es la base para calcular el **coeficiente de clustering** de un nodo o de toda la red
- Paso fundamental previo a: detección de comunidades, cálculo de k-trusses y detección de spam

$$C_i = \frac{2 \times \text{triángulos que incluyen a } i}{k_i(k_i - 1)}$$

donde k_i es el grado del nodo i .

El conteo de triángulos se considera el primer paso fundamental para calcular métricas de red más avanzadas como transitividad y comunidad (arXiv, s.f.-a; arXiv, s.f.-b).

SECCIÓN 3

Analítica de Datos de Texto

Preprocesamiento, representación vectorial y modelado de tópicos

Analítica de Texto y PLN

Fundamentos y aplicaciones

- El **Procesamiento de Lenguaje Natural (PLN)** permite a las máquinas interpretar y generar texto humano
- Aplicaciones: análisis de sentimientos, clasificación de documentos, chatbots, extracción de información, modelado de tópicos
- El texto es **no estructurado**: requiere transformarse a una representación numérica antes de aplicar algoritmos de ML

La analítica de texto combina lingüística computacional y estadística para extraer valor de grandes volúmenes de texto (UniSabana, s.f.; OpenWebinars, s.f.).

Preprocesamiento de Texto

Tokenización, stopwords y normalización

Pipeline típico

1. **Minúsculas:** normalizar mayúsculas/minúsculas
2. **Tokenización:** dividir el texto en unidades (tokens) — palabras u oraciones
3. **Eliminación de stopwords:** quitar palabras muy frecuentes sin valor semántico ("el", "de", "y")
4. **Eliminación de puntuación, números y símbolos especiales**
5. **Normalización léxica:** stemming o lematización

Un buen preprocesamiento reduce el ruido y la dimensionalidad del vocabulario antes de vectorizar (Q2BStudio, s.f.; datitos, s.f.).

```
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

texto = "Los clientes ESTÁN comprando productos similares!"
texto = texto.lower()
texto = re.sub(r"^\w\s]", "", texto)
tokens = texto.split()
tokens = [t for t in tokens if t not in stopwords.words("spanish")]

lemmatizer = WordNetLemmatizer()
tokens = [lemmatizer.lemmatize(t) for t in tokens]
print(tokens)
```

Stemming vs. Lematización

Dos formas de normalizar palabras

Stemming

- Recorta la palabra hasta una **raíz** (no necesariamente una palabra real)
- Más rápido, pero menos preciso
- Ej: "corriendo" → "corr"

Lematización

- Reduce la palabra a su **lema** (forma canónica válida), respetando la categoría gramatical
- Más preciso, pero computacionalmente más costoso
- Ej: "corriendo" → "correr"

La elección entre stemming y lematización depende del equilibrio deseado entre velocidad y precisión semántica (Q2BStudio, s.f.).

Representación Vectorial

Bag of Words (BoW)

- Cada documento se representa como un vector de **conteo de palabras**
- Ignora orden y gramática
- Vectores dispersos y de alta dimensión

TF-IDF

$$\text{tfidf}(t, d) = \text{tf}(t, d) \times \log \frac{N}{df(t)}$$

- Pondera cada término por su frecuencia en el documento y su rareza en el corpus
- Reduce el peso de palabras muy comunes en todos los documentos

TF-IDF mejora sobre BoW puro al destacar términos discriminativos y atenuar los omnipresentes (Pedromebo, s.f.; Murzone, s.f.).

```
from sklearn.feature_extraction.text import TfidfVectorizer

corpus = ["el gato come pescado", "el perro come carne", "el gato duerme"]

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(corpus)

print(vectorizer.get_feature_names_out())
print(X.toarray())
```

Word Embeddings

Word2Vec y similitud semántica

- Representan cada palabra como un **vector denso** en un espacio de alta dimensión
- Palabras que aparecen en **contextos similares** obtienen vectores cercanos (hipótesis distribucional)
- A diferencia de BoW/TF-IDF, capturan relaciones **semánticas y sintácticas**, no solo frecuencia

Word2Vec (Google, 2013) aprende embeddings prediciendo palabras a partir de su contexto (o viceversa), útil como entrada para tareas como análisis de sentimientos (iAyProgramación, s.f.; Pedromebo, s.f.).

Modelado de Tópicos con LDA

Latent Dirichlet Allocation

Idea generativa

- Cada **documento** es una mezcla de **tópicos**
- Cada **tópico** es una mezcla de **palabras**
- Proceso generativo: para cada documento se muestrea una distribución de tópicos; para cada palabra se muestrea un tópico y luego una palabra de ese tópico

LDA usa inferencia bayesiana (Gibbs sampling o variational Bayes) para "revertir" el proceso generativo y descubrir los tópicos latentes a partir de las palabras observadas (Blei et al., 2003, según Wikipedia, s.f.; IBM, s.f.).

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.decomposition import LatentDirichletAllocation

vectorizer = CountVectorizer(max_df=0.9, min_df=2, stop_words="spanish")
X = vectorizer.fit_transform(documentos)

lda = LatentDirichletAllocation(n_components=5, random_state=42)
lda.fit(X)

for i, topic in enumerate(lda.components_):
    palabras = [vectorizer.get_feature_names_out()[j]
                 for j in topic.argsort()[-10:]]
    print(f"Tópico {i}: {palabras}")
```

La visualización interactiva con `pyLDavis` ayuda a interpretar la distancia y solapamiento entre tópicos (GeeksforGeeks, s.f.).

Referencias

Fuentes bibliográficas

- AltexSoft. (s.f.). *NoSQL databases visually explained with examples*.
<https://www.altexsoft.com/blog/nosql-databases/>
- AprendeIA. (s.f.). *Algoritmo Apriori – Teoría*. <https://aprendeia.com/algoritmo-apriori/>
- arXiv. (s.f.-a). *Fast triangle counting*. <https://arxiv.org/pdf/2309.09064>
- arXiv. (s.f.-b). *Triangle counting accelerations: From algorithm to in-memory computing architecture*. <https://arxiv.org/pdf/2112.00471>
- BlazeClan. (s.f.). *4 types of NoSQL databases*. <https://blazeclan.com/blog/dive-deep-types-nosql-databases/>